Research Topics in Blockchain
&
Blockchain and Its Application



Lecture # 7:

- ✓ Revision of Goofy Coin
- ✓ Goofy Coin
- ✓ Workflow of Goofy Coin
- ✓ Scrooge coin
- ✓ Workflow of Scrooge coin
- ✓ Challenges
- ✓ Chapter 1 Question
- ✓ Start of Chapter 2



GoofyCoin

Goofy can create new coins

Transaction =(Signature)||(M)
For Spending the Message should encrypt

signed by pk_{Goofy}

CreateCoin [uniqueCoinID]

Transaction =(Signature)||(M)

New coins belong to me.



A coin's owner can spend it.

Transaction =(Signature)||(M)
Encrypt with Public Key of Alice

signed by pk_{Goofy}

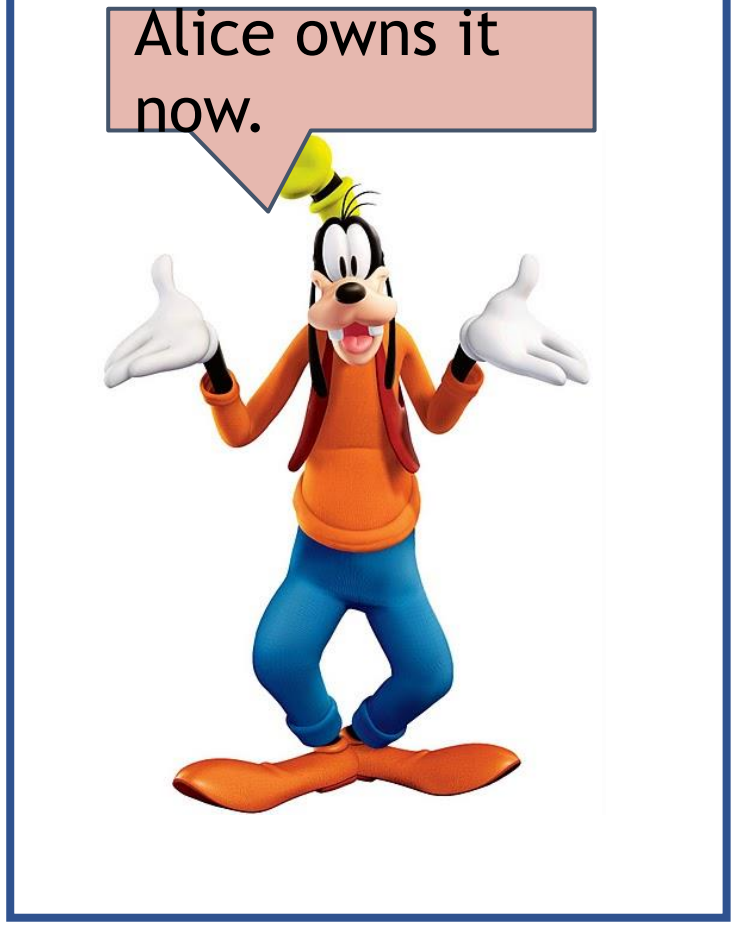
Pay to pk_{Alice} : H( )

Hash of Previous Transaction

CreateCoin [uniqueCoinID]

Genesis Block

ransaction =(Signature)||(M)



```python
from cryptography.fernet import Fernet
import hashlib

# Step 1: Generate a Fernet Key
fernet_key = Fernet.generate_key()
cipher = Fernet(fernet_key)

# Step 2: Original Message
message = b"Hello"

# Step 3: Create a SHA-256 Hash (Signature)
hashed_message = hashlib.sha256(message).hexdigest().encode()

# Step 4: Append Signature to Message
signed_message = message + b"||" + hashed_message  # Separator for clarity

# Step 5: Encrypt the Signed Message
encrypted_message = cipher.encrypt(signed_message)
print("Encrypted Message:", encrypted_message)

# Step 6: Decrypt the Message
decrypted_message = cipher.decrypt(encrypted_message)

# Step 7: Remove the Signature and Extract Original Message
original_message, extracted_signature = decrypted_message.split(b"||")

print("Decrypted Message:", original_message.decode())  # Output: Hello
```

The recipient can pass on the coin again.

signed by pk_{Alice}

Pay to pk_{Bob} : H( )

Transaction =(Signature)||(M)
Encrypt with Public Key of Alice

signed by pk_{Goofy}

Pay to pk_{Alice} : H( )

Hash(Previous Transaction)

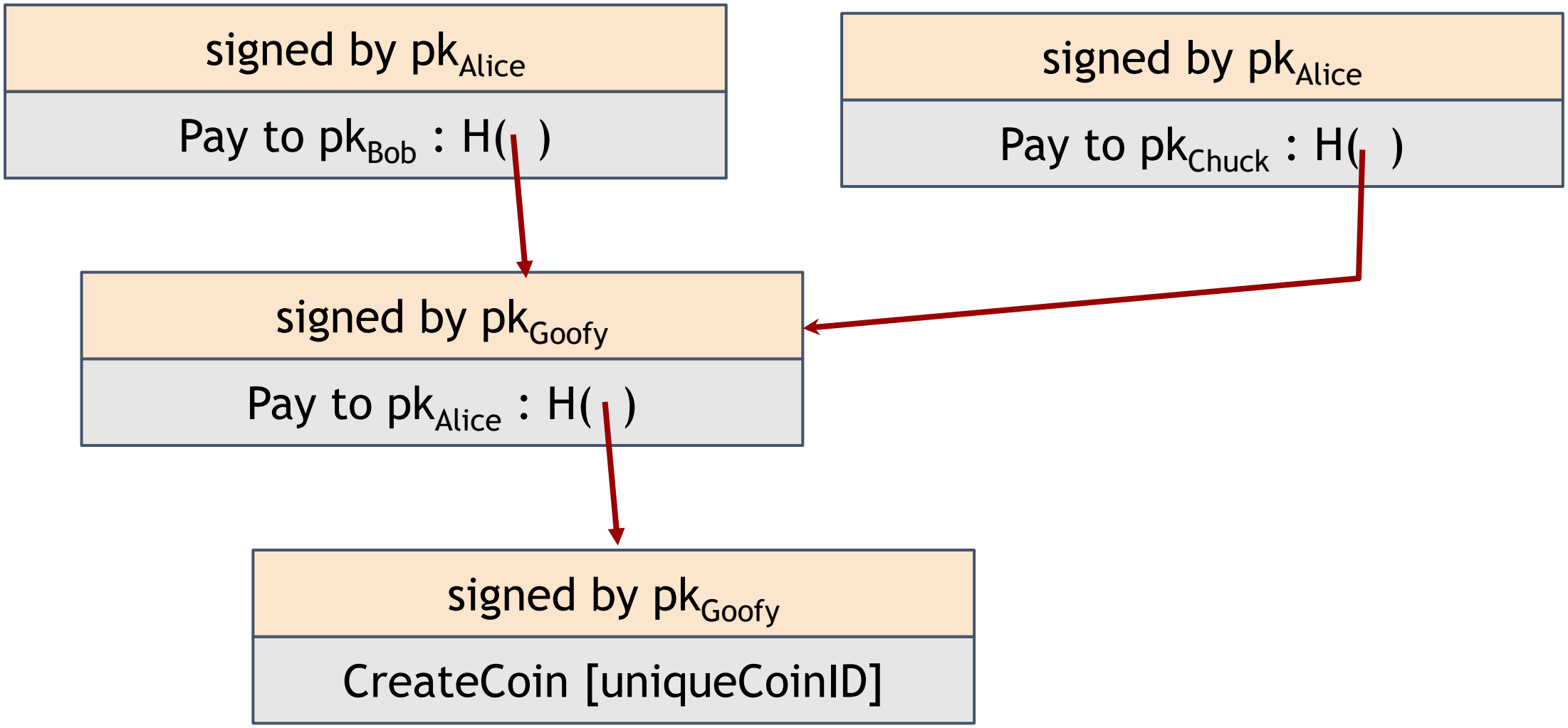
signed by pk_{Goofy}

Genesis Block

Transaction
(Signature)||(
M)



double-spending attack



double-spending attack

the main design challenge in digital currency

- The first key idea is that a designated entity called Scrooge publishes an append-only ledger containing the history of all the transactions that have happened.
- The append-only property ensures that any data written to this ledger will remain forever.
- If the ledger is truly append-only, we can use it to defend against double-spending by requiring all transactions to be written the ledger before they are accepted.



ScroogeCoi

- To implement this append-only functionality, Scrooge can build a block chain (the data structure we discussed before) which he will digitally sign.
- It's a series of data blocks, each with one transaction in it (in practice, as an optimization, we'd really put multiple transactions into the same block, as Bitcoin does.) Each block has the ID of a transaction, the transaction's contents, and a hash pointer to the previous block.
- Scrooge digitally signs the final hash pointer, which binds all of the data in this entire structure, and publishes the signature along with the block chain.



ScroogeCoin

Scrooge publishes a history of all transactions
(a block chain, signed by Scrooge)



Genisus Block

prev: H( )
transID: 71
Transaction Signed by the owner ,and the Scoocge

H(Transactio n)
transID: 72
Transaction Signed by the owner ,and the Scoocge

H(Transactio n)
transID: 73
Transaction Signed by the owner ,and the Scoocge

Signed by Goofy

optimization: put multiple transactions in the

CreateCoins transaction creates new coins

transID: 73	type:CreateCoins	
coins created		
num	*value*	*recipient*
0	3.2	0x…
1	1.4	0x…
2	7.1	0x…

coinID 73(0)

coinID 73(1)

coinID 73(2)

Valid, because I said so.



Concept of Centralizim

PayCoins transaction consumes (and destroys) some coins, and creates new coins of the same total value

transID: 73 type:PayCoins		
consumed coinIDs: 68(1), 42(0), 72(3)		
coins created		
num	*value*	*recipient*
0	3.2	0x...
1	1.4	0x...
2	7.1	0x...
signatures		

Important Point
Status of coin
redeam and spend

Valid if:
-- consumed coins valid,
-- not already consumed,
-- total value out = total value in,
and
-- signed by owners of all consumed coins

The rules of Scrooge Coin say that Pay Coins transaction is valid if four things are true:

- The consumed coins are valid, that is, they really were created in previous transactions.
- The consumed coins were not already consumed in some previous transaction. That is, that this is not a double-spend.
- The total value of the coins that come out of this transaction is equal to the total value of the coins that went in. That is, only Scrooge can create new value.
- The transaction is validly signed by the owners of all of the consumed coins.



Coins in this system are immutable

- They are never changed, subdivided, or combined. Each coin is created, once, in one transaction and later consumed in some other transaction.
- But we can get the same effect as being able to subdivide or combine coins by using transactions.
- For example, to subdivide a coin, Alice create a new transaction that consumes that one coin, and then produces two new coins of the same total value.
- Those two new coins could be assigned back to her. So although coins are immutable in this system, it has all the flexibility of a system that didn't have immutable coins.





```
coin_1 = {
    "value": "10 Rupees",
    "material": "Bi-metallic",
    "weight": "7.74g",
    "diameter": "27mm",
    "mint_year": "2024",
    "security_features": "RBI-approved"
}
```

```
coin_2 = {
    "value": "10 Rupees",  # Value is the same
    "material": "Bi-metallic",
    "weight": "7.74g",
    "diameter": "27mm",
    "mint_year": "2023",  # Different mint year
    "security_features": "New RBI-approved design"
}
```

Hash function properties (again). Let H be a hash function that is both hiding and puzzle-friendly. Consider G(z) = H(z) || zlast where zlast represents the last bit of z. Show that G is puzzle-friendly but not hiding.

Proving Puzzle-Friendliness

A hash function H is **puzzle-friendly** if, given an output y, it is computationally infeasible to find an input z such that $H(z) = y$, except by trying many different values of z.

Since $G(z)$ is constructed as:

$$G(z) = H(z) \parallel z_{\text{last}}$$

The function still contains $H(z)$, which is puzzle-friendly by assumption.

The additional bit z_{last} does not affect the difficulty of finding z, as brute-force attempts will still require finding a preimage of $H(z)$.

Given $G(z)$, an attacker still needs to solve the hard problem of inverting $H(z)$, making G puzzle-friendly.

Thus, G retains the puzzle-friendliness of H.

2. Proving Non-Hiding Property

A hash function H is **hiding** if, given $H(z)$, it is computationally infeasible to deduce anything about z. That is, if z is chosen from a sufficiently large space, its value should remain indistinguishable.

However, in the case of $G(z) = H(z) \parallel z_{\text{last}}$:

- The function leaks **one bit** of z explicitly (i.e., z_{last}).
- This violates the hiding property because an adversary who sees $G(z)$ immediately learns whether z is odd or even (i.e., the value of z_{last}).
- If z were a secret, this leakage would already reveal partial information, breaking the hiding property.

Mathematically, if z is drawn from a uniform distribution, then:

$$P(Z = z|G(Z))$$

is greater than $P(Z = z)$, meaning $G(z)$ gives additional information about z, breaking the hiding property.

In ScroogeCoin, suppose Mallory tries generating (sk, pk) pairs until her secret key matches someone else's. What will she be able to do?

If Mallory finds a secret key sksksk that matches someone else's (e.g., Alice's), she can sign transactions on behalf of that person.

- This would allow her to **spend Alice's coins without permission**, effectively performing a key collision attack.

What if Alice's random number generator has a bug and her key generation procedure produces only 1,000 distinct pairs?

- The search space is drastically reduced to **1,000 keys**.
- By brute force, Mallory only needs to try at most **1,000 attempts** to find a match.
- This makes key collision **trivial**, completely breaking Alice's security.



Crucial question:

Can we descroogify the currency, and operate without any central,

Problem Statement:

In a centralized banking system for a mobile application, users can initiate financial transactions such as fund transfers, bill payments, and online purchases. However, due to network delays and concurrent transaction requests, there is a possibility that the same balance might be processed multiple times before being updated in the system. This can lead to inconsistencies in the ledger, allowing an unauthorized execution of multiple transactions beyond the actual account balance.To ensure transaction integrity and prevent unauthorized duplicate deductions, a robust mechanism is required that maintains an immutable and sequentially linked record of transactions, ensuring real-time validation of account balances.

Chapter 2: How Bitcoin Achieves Decentralization

In this chapter, we will discuss decentralization in Bitcoin. In the first chapter we looked at the crypto basics that underlie Bitcoin and we ended with a simple currency that we called ScroogeCoin. ScroogeCoin achieves a lot of what we want in a ledger-based cryptocurrency, but it has one glaring problem — it relies upon the centralized authority called Scrooge. We ended with the question of how to decentralize, or de-Scrooge-ify, this currency, and answering that question will be the focus of this chapter.

As you read through this chapter, take note that the mechanism through which Bitcoin achieves decentralization is not purely technical, but it's a combination of technical methods and clever incentive engineering. At the end of this chapter you should have a really good appreciation for how this decentralization happens, and more generally how Bitcoin works and why it is secure.

Centralization Disadvantage

- ✓ Single entity to maintains the ledger
- ✓ Single entity authority over each transactions for validation
- ✓ Single entity to creates new bitcoins
- ✓ Single entity responsible to determines the rules of the system change



How Bitcoin Achieves Decentralization

Decentralization in Bitcoin ??

1. Who maintains the ledger?
2. Who has authority over which transactions are valid?
3. Who creates new bitcoins?
4. Who determines how the rules of the system change?
5. How do bitcoins acquire exchange value?

